Murzeau Jérémy

TP : Installation, Configuration et Sécurisation MQTT sur Ubuntu

1. Déploiement via Docker

```
brabi@DESKTOP-EH29IVM:~/IoT/tp2/docker_basic$ docker-compose up -d
Starting mosquitto_secure ... done
brabi@DESKTOP-EH29IVM:~/IoT/tp2/docker_basic$ docker-compose ps
      Name                    Command               State            Ports
-------------------------------------------------------------------------------
mosquitto_secure   /docker-entrypoint.sh     Up      1883/tcp, 0.0.0.0:8883-
                   /usr ...                         >8883/tcp,:::8883-
                                                    >8883/tcp
```

2. Sécurisation

a. Communication sécurisée MQTTS : chiffrement TLS et authentification

Le certificat CA sécurise, les identifiants valident l'accès utilisateur.

```
brabi@DESKTOP-EH29IVM:~$ sudo mosquitto -c /etc/mosquitto/mosquitto.conf -v
1774894712: Info: running mosquitto as user: mosquitto.
1774894712: mosquitto version 2.1.2 starting
1774894712: Config loaded from /etc/mosquitto/mosquitto.conf.
1774894712: Bridge support available.
1774894712: Persistence support available.
1774894712: TLS support available.
1774894712: TLS-PSK support available.
1774894712: Websockets support available.
1774894712: Warning: File /etc/mosquitto/passwd has world readable permissio
ns. Future versions will refuse to load this file.
To fix this, use `chmod 0700 /etc/mosquitto/passwd`.

1774894712: Plugin builtin-security has registered to receive 'basic-auth' e
vents.
1774894712: Warning: File /etc/mosquitto/acl has world readable permissions.
 Future versions will refuse to load this file.
To fix this, use `chmod 0700 /etc/mosquitto/acl`.

1774894712: Plugin builtin-security has registered to receive 'acl-check' ev
ents.
1774894712: Opening ipv4 listen socket on port 8883.
1774894712: Opening ipv6 listen socket on port 8883.
1774894712: mosquitto version 2.1.2 running
1774894765: New connection from 127.0.0.1:60540 on port 8883.
1774894765: New client connected from 127.0.0.1:60540 as auto-135B2D70-845D-
7A8B-CED6-85F75B6EDCBE (p4, c1, k60, u'sensor_node_1').
1774894765: No will message specified.
1774894765: Client auto-135B2D70-845D-7A8B-CED6-85F75B6EDCBE negotiated TLSv
1.3 cipher TLS_AES_256_GCM_SHA384
1774894765: Sending CONNACK to auto-135B2D70-845D-7A8B-CED6-85F75B6EDCBE (0,
 0)
1774894765: Received PUBLISH from auto-135B2D70-845D-7A8B-CED6-85F75B6EDCBE
(d0, q0, r0, m0, 'home/sensor1/temperature', ... (4 bytes))
1774894765: Received DISCONNECT from auto-135B2D70-845D-7A8B-CED6-85F75B6EDC
BE
1774894765: Client auto-135B2D70-845D-7A8B-CED6-85F75B6EDCBE [127.0.0.1:6054
0] disconnected.
```

```
brabi@DESKTOP-EH29IVM:~$ ls
 IoT  'VirtualBox VMs'   VirtualBox_Images   server.crt
brabi@DESKTOP-EH29IVM:~$ cd IoT/
brabi@DESKTOP-EH29IVM:~/IoT$ ls
formation-Jour1  formation-Jour2  tp2
brabi@DESKTOP-EH29IVM:~/IoT$ cd tp2/
brabi@DESKTOP-EH29IVM:~/IoT/tp2$ ls
docker_basic
brabi@DESKTOP-EH29IVM:~/IoT/tp2$ mosquitto_pub -h localhost -p 8883 \
--cafile /etc/mosquitto/certs/ca.crt \
-u sensor_node_1 -P toto \
-t "home/sensor1/temperature" -m "22.5"
brabi@DESKTOP-EH29IVM:~/IoT/tp2$
```

b. Contrôle d'accès (ACL) : échec d'une tentative de publication non autorisée

L'utilisateur dashboard est bien authentifié, mais le broker refuse sa publication car il n'en a pas l'autorisation.

```
1774894712: Persistence support available.
1774894712: TLS support available.
1774894712: TLS-PSK support available.
1774894712: Websockets support available.
1774894712: Warning: File /etc/mosquitto/passwd has world readable permissio
ns. Future versions will refuse to load this file.
To fix this, use `chmod 0700 /etc/mosquitto/passwd`.

1774894712: Plugin builtin-security has registered to receive 'basic-auth' e
vents.
1774894712: Warning: File /etc/mosquitto/acl has world readable permissions.
 Future versions will refuse to load this file.
To fix this, use `chmod 0700 /etc/mosquitto/acl`.

1774894712: Plugin builtin-security has registered to receive 'acl-check' ev
ents.
1774894712: Opening ipv4 listen socket on port 8883.
1774894712: Opening ipv6 listen socket on port 8883.
1774894712: mosquitto version 2.1.2 running
1774894765: New connection from 127.0.0.1:60540 on port 8883.
1774894765: New client connected from 127.0.0.1:60540 as auto-135B2D70-845D-
7A8B-CED6-85F75B6EDCBE (p4, c1, k60, u'sensor_node_1').
1774894765: No will message specified.
1774894765: Client auto-135B2D70-845D-7A8B-CED6-85F75B6EDCBE negotiated TLSv
1.3 cipher TLS_AES_256_GCM_SHA384
1774894765: Sending CONNACK to auto-135B2D70-845D-7A8B-CED6-85F75B6EDCBE (0,
 0)
1774894765: Received PUBLISH from auto-135B2D70-845D-7A8B-CED6-85F75B6EDCBE
(d0, q0, r0, m0, 'home/sensor1/temperature', ... (4 bytes))
1774894765: Received DISCONNECT from auto-135B2D70-845D-7A8B-CED6-85F75B6EDC
BE
1774894765: Client auto-135B2D70-845D-7A8B-CED6-85F75B6EDCBE [127.0.0.1:6054
0] disconnected.
1774894893: New connection from 127.0.0.1:59788 on port 8883.
1774894893: Sending CONNACK to auto-2F915AC4-4AB8-2F7F-6F02-33A68A26218D (0,
 5)
1774894893: Client auto-2F915AC4-4AB8-2F7F-6F02-33A68A26218D [127.0.0.1:5978
8] disconnected: not authorised.
```

```
brabi@DESKTOP-EH29IVM: $ ls
 IoT  'VirtualBox VMs'   VirtualBox_Images   server.crt
brabi@DESKTOP-EH29IVM: $ cd IoT/
brabi@DESKTOP-EH29IVM:~/IoT$ ls
formation-Jour1  formation-Jour2  tp2
brabi@DESKTOP-EH29IVM:~/IoT$ cd tp2/
brabi@DESKTOP-EH29IVM:~/IoT/tp2$ ls
docker_basic
brabi@DESKTOP-EH29IVM:~/IoT/tp2$ mosquitto_pub -h localhost -p 8883 \
--cafile /etc/mosquitto/certs/ca.crt \
-u sensor_node_1 -P toto \
-t "home/sensor1/temperature" -m "22.5"
brabi@DESKTOP-EH29IVM:~/IoT/tp2$ mosquitto_pub -h localhost -p 8883 \
--cafile /etc/mosquitto/certs/ca.crt \
-u dashboard -P toto \
-t "home/sensor1/temperature" -m "Hack"
Connection error: Connection Refused: not authorised
Error: The connection was refused
brabi@DESKTOP-EH29IVM:~/IoT/tp2$
```

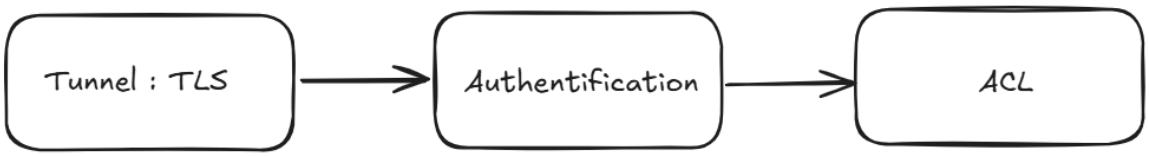
Le broker n'accepte plus de flux qui ne sont pas sécurisés ni les anonymes.

```
brabi@DESKTOP-EH29IVM:~/IoT/tp2/docker_basic$ mosquitto_pub -h localhost -t
test -m 'anon'
Error: Bad file descriptor
```

3. Questions

- *Pour chaque risque identifié/vulnérabilité, proposez au moins une mesure de mitigation concrète* :
  - Accès anonyme : n'importe qui peut se connecter au broker. Pour corriger, il faut forcer l'authentification en passant l'option *allow_anonymous* à *false*.

Pour qu'il y ait obligatoirement besoin d'un nom d'utilisateur et d'un mot de passe.
    - Par défaut, MQTT envoie tout en clair sur le port 1883, les informations sont donc lisibles par n'importe qui. Il faut donc fermer ce port et activer le TLS sur le port 8883. Les données sont alors cryptées.
    - Si tous les utilisateurs ont tous les droits, un capteur piraté pourrait envoyer des ordres malveillants à autre appareil. Pour éviter ça, il faut mettre en place des ACL, où chaque utilisateur est limité à ses propres topics.
    - Docker permet d'isoler Mosquitto, si le service est attaqué, l'attaquant reste bloqué dans le conteneur sans pouvoir toucher au reste du système.
- *Quels sont les principaux challenges ou obstacles rencontrés pour leur mise en œuvre* ?
    - La gestion des certificats (TLS), il faut s'assurer que le client possède bien le bon certificat. J'ai rencontré des erreurs de connexion parce que le chemin vers ces fichiers n'était pas le bon ou parce que le nom d'hôte ne correspondait pas à ce qui était écrit dans le certificat.
    - La configuration des volumes Docker, pour ne pas se tromper dans le chemin du fichier, Docker n'a pas trouvé la configuration.
    - Les conflits de ports, car le port 8883 était déjà utilisé par une autre instance de Mosquitto qui tournait en arrière-plan sur le PC. Il faut penser à arrêter arrêter les services.

- *Quels risques si MQTT non sécurisé ?*
    - Etant donné que les messages circulent en clair, un attaquant sur le réseau peut lire les informations envoyées.
    - Un attaquant peut essayer de se faire passer pour un capteur et envoyer de mauvaises données au système, ou prendre le contrôle en envoyant ses propres messages.
- *Différence TLS/mTLS ?*
    - Dans le TLS, le client vérifie que le serveur est bien le bon grâce à son certificat, mais le serveur, ne vérifie pas l'identité du client (en général un mot de passe).
    - Dans le mTLS, les deux parties s'authentifient avant de pouvoir discuter.
- *Pourquoi ACL ?*
    - Pour savoir ce qu'un utilisateur a le droit de faire. Cela permet en particulier d'appliquer le principe du moindre privilège, pour que n'importe qui ne puisse pas lire ou modifier n'importe quel topic. Cela peut aussi servir à bloquer un attaquant sur des topics spécifiues à un capteur.

4. Schéma technique :